

Appunti Sistemi Operativi – Laboratorio

Sommario

Capitolo 1 – Introduzione a Unix.....	2
Panoramica su UNIX.....	2
Architettura di UNIX	3
Autenticazione	4
Identificazione degli Utenti	4
User ID e Superutente	4
Logging in	5
File e Directory	5
Esempio: Elencare i file di una directory (comando ls).....	5
Input/Output.....	6
I/O non bufferizzato.....	6
Esempio: Leggere dallo std input e scrivere sullo std output (I/O NON Bufferizzato)	6
Standard I/O	7
Esempio: Leggere dallo std input e scrivere sullo std output (I/O Bufferizzato)	7
Programmi e Processi.....	8
Esempio.....	8
Controllo dei processi	8
Esempio programma che legge i comandi dallo standard input e li esegue	9
Gestione degli errori.....	10
Esempio.....	10
Segnali.....	11
Chiamate di sistema.....	11
Funzioni di libreria e kernel.....	11
Capitolo 2 – Programmazione in C in ambiente Unix/Linux.....	14
Compilazione in ambiente Unix/Linux.....	14
Opzioni comuni del compilatore.....	14
Compilazione e costruzione di un programma.....	14
Codice oggetto e linking	15
Compilazione e Linking.....	15
Debug	16
Organizzazione del codice	16
Compilazione separata	17
MakeFile	17
Funzionamento informale di make.....	17
Struttura di un Makefile	18
Regole del makefile	18
Costruzione dell'albero delle dipendenze.....	18
Regole predefinite.....	18
Makefile parametrico	19
Esempio pratico: usaPal	19
Makefile tipico.....	20
Compressione dei File.....	21
Capitolo 3 – Introduzione alla shell UNIX	22
Definizione e Funzionalità di una Shell.....	22
Le Shell di Unix.....	22
La Shell di Linux.....	23
Usi della shell: interprete dei comandi	23

Startup del sistema e shell di Login.....	23
Parsing della riga di comando	23
Tipi di comandi.....	23
I Processi e la shell	24
Proprietà e Permessi	24
Maschera di creazione dei file	25
Cambiare permessi e proprietà	25
Comando su	26
Directory di lavoro.....	26
Variabili	27
Redirezione	27
Pipe.....	28
Bourne Again Shell.....	28
I comandi della shell	29
Comando history.....	29
Altri comandi.....	29
Visualizzare il contenuto di un file	30
Alias.....	30
Comandi frequenti per la gestione delle directory	30
Primi comandi	30
Inviare segnali ad un processo.....	31
kill.....	31
Esempio: kill.....	32
Processi foreground	32
Processi background	32
Link	33
Tipologie di link	33
Esempio di creazione link	33

Capitolo 1 – Introduzione a Unix

Panoramica su UNIX

UNIX rappresenta un modello paradigmatico per comprendere i servizi di base forniti da un **sistema operativo**, quali l'**autenticazione**, la gestione di **file** e **directory**, i meccanismi di **input/output**, la gestione di **programmi** e **processi**, gli strumenti per la **gestione degli errori**, l'uso dei **segnali** e infine le **chiamate di sistema** e le **funzioni di libreria**. Tutti i sistemi operativi forniscono servizi essenziali per l'esecuzione di programmi, l'accesso ai file e la gestione della memoria, ma l'obiettivo è descrivere tali servizi nel contesto di **UNIX**, a partire da una **panoramica storica**.

UNIX è un sistema operativo **multiutente** e **multitasking**, che consente a più utenti di eseguire simultaneamente vari **processi**. Nasce anche come **ambiente di sviluppo software**, offrendo strumenti come **editor**, **compilatori** e **interpreti**. È stato progettato per essere **portabile** e **indipendente dall'hardware**, grazie all'uso del linguaggio di programmazione **C**, rendendolo facilmente adattabile a diversi **sistemi**.

Nel 1964, **AT&T**, **MIT** e **General Electric** collaborano alla creazione di un sistema operativo innovativo chiamato **MULTICS**, pensato per essere **multiutente** e **multiprocessore**. Tuttavia, nel 1969, il progetto fallisce. Dai suoi resti nasce **UNIX**, sviluppato ai **Bell Labs** da **Ken Thompson** e **Dennis Ritchie**. Basandosi su **MULTICS**, **UNIX** supera molte delle limitazioni dei sistemi precedenti, mantenendo le caratteristiche di **multiutenza** e **multitasking**. Il nome **UNIX** fu scelto in contrapposizione a **MULTICS**. Nel 1973, la terza versione di **UNIX** viene riscritta in **linguaggio C**, sviluppato da **Dennis Ritchie** come evoluzione del linguaggio **B**. Questo passaggio segna un punto cruciale, perché dimostra che anche i **linguaggi ad alto livello** possono

garantire buone **prestazioni**. Alla fine degli anni '70, **AT&T** decide di rendere **UNIX** disponibile alle **università**, distribuendone i **sorgenti** a basso costo, una scelta dettata anche da vincoli **antitrust**.

Nel 1984 nasce presso l'**Università di Berkeley** la distribuzione **BSD (Berkeley Software Distribution)**, finanziata dal **DARPA** per creare uno **standard UNIX** a uso governativo. Una delle versioni più influenti è la **4.3BSD**, inizialmente per **VAX**, successivamente portata su molte piattaforme. Sempre nel 1984, molte delle funzionalità di **BSD** vengono integrate nella versione **System V** di **AT&T**. Oggi esistono diverse implementazioni di **UNIX**, supportate da vari produttori come **Sun Microsystems (SunOS/Solaris)**, **Microsoft (XENIX)**, **IBM (AIX)** e altri.

Solaris è la versione di **UNIX** sviluppata da **Sun Microsystems**, basata su **System V Release 4** con oltre dieci anni di miglioramenti. Rappresenta l'unico caso di successo **commerciale** derivato da **SVR4** formalmente certificato come sistema **UNIX**, con la sua ultima versione identificata come **Solaris 10**. Anche se **Solaris** è nato come **software proprietario**, dal 2005 parte del suo **codice** è stato rilasciato con **licenza open source CDDL**, dando vita al progetto **OpenSolaris**, che diventerà la base delle versioni future.

Nel 1983, **Richard Stallman** del **MIT** avvia la re-implementazione libera di **UNIX** con il progetto **GNU**, includendo strumenti fondamentali come **emacs**, **gcc** e **gdb**. Nel 1991, **Linus Torvalds**, studente finlandese, realizza il **kernel Linux**, compatibile con **POSIX**, che combinato con i tool **GNU** dà origine a un sostituto completo di **UNIX**. Sempre nel 1991, l'**Università di Berkeley** pubblica una versione completamente **free** di **UNIX**, rimuovendo il **codice proprietario AT&T** e aprendo la strada a progetti indipendenti come **FreeBSD**, **NetBSD** e **OpenBSD**.

La **progettazione** di **UNIX** è stata fortemente influenzata dai limiti dell'**hardware** dell'epoca, come **telescriventi** e **terminali video** a 80 caratteri. Questo ha portato alla definizione di **comandi** e **sintassi** essenziali, con ciascun programma focalizzato su una sola funzione, da eseguire in modo **semplice** ed **efficiente**. L'**output** di un comando può essere riutilizzato come **input** per un altro, permettendo combinazioni potenti attraverso la **shell**, che non richiede **privilegi speciali**. Tutto questo favorisce l'**automazione** delle attività di routine.

UNIX ha ottenuto successo grazie alla sua **dimensione contenuta**, alla **modularità** e al **design pulito**. È scritto in **linguaggio C**, il che garantisce **portabilità** tra hardware diversi. Le **applicazioni UNIX** possono essere adattate con facilità a nuove piattaforme tramite il semplice **ricompilamento del codice sorgente**. Inoltre, è un ambiente molto **produttivo** per lo sviluppo software, con una vasta disponibilità di **strumenti** e una **sintassi dei comandi** versatile. La sua diffusione è anche frutto di un processo di **serendipità**.

Durante gli anni '80, la grande varietà di versioni di **UNIX** ha portato alla necessità di una **standardizzazione**, promossa dal **governo USA**. Tra i principali **standard** troviamo il **linguaggio C**, definito nel 1989 da **ANSI** e successivamente da **ISO**, pensato per favorire la **portabilità del codice**. Il progetto **POSIX**, nato nel 1988, definisce un'interfaccia coerente tra sistemi. Seguono gli **standard XPG** di **X/Open** e **SVID** di **AT&T**, entrambi sviluppati nel 1989 per migliorare l'**interoperabilità** tra le diverse implementazioni **UNIX**.

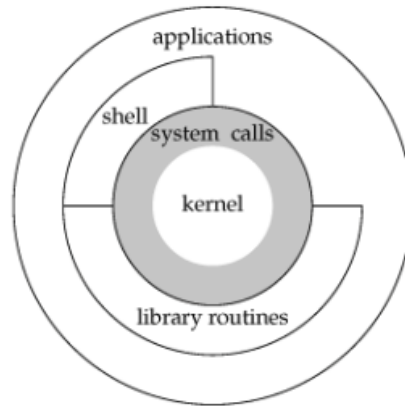
Lo **standard POSIX.1**, introdotto nel 1990, definisce l'**interfaccia di programmazione** tra applicazioni e sistema operativo, con l'obiettivo di garantire la **portabilità del codice sorgente**. Nel 1992, **POSIX.2** stabilisce il **linguaggio della shell** e i **comandi principali**, assicurando che gli **script** siano eseguibili su qualsiasi sistema conforme a **POSIX**.

Architettura di UNIX

Un **sistema operativo (SO)** può essere considerato come un software che gestisce le **risorse hardware** del computer e crea un **ambiente di esecuzione** per i programmi. Questo componente centrale del sistema è chiamato **kernel**, ed è tipicamente di dimensioni **ridotte**, pur essendo fondamentale per il funzionamento del sistema.

Il **kernel** non interagisce direttamente con le applicazioni, ma lo fa tramite uno strato software intermedio noto come **system call** o **chiamate di sistema**. Su questo strato si costruiscono le **librerie di funzioni**, che offrono strumenti comuni e semplificano l'interazione con il sistema operativo.

Un elemento fondamentale dell'architettura è la **shell**, un'applicazione speciale che funge da **interfaccia tra l'utente e il kernel**, permettendo l'avvio e la gestione di altri programmi.



La struttura di UNIX può essere visualizzata come una serie di **strati concentrici**. Al centro si trova il **kernel**, circondato dallo strato delle **chiamate di sistema**. Subito sopra si collocano le **librerie di sistema** e la **shell**, che consentono l'interazione tra l'utente e il cuore del sistema. Infine, all'esterno, troviamo le **applicazioni utente**, che sfruttano questi strati per accedere ai servizi forniti dal sistema operativo.

Autenticazione

Quando si accede a un sistema **UNIX**, è necessario fornire un login e una **password**. Questi dati vengono verificati attraverso il file di sistema **/etc/passwd**, dove ogni riga rappresenta un utente. Ogni riga è composta da **sette campi**, tra cui il **nome utente**, la **password cifrata**, l'**ID utente** e l'**ID del gruppo**, un **commento descrittivo**, la **home directory** e il percorso alla **shell** utilizzata.

Un esempio di riga nel file **/etc/passwd** potrebbe essere:

`staiano:x:500:500: Antonino Staiano:/home/staiano:/bin/bash`, che fornisce informazioni dettagliate sull'utente, inclusi gli ID numerici e il tipo di shell.

Identificazione degli Utenti

Ogni **utente** è associato a un **gruppo**, identificato da un **group ID**. È possibile che più utenti appartengano allo **stesso gruppo**, come ad esempio studenti, docenti o ospiti. I **gruppi** servono a gestire la **condivisione delle risorse** tra più utenti. Le informazioni relative alla corrispondenza tra ID numerici e gruppi si trovano solitamente nel file di configurazione **/etc/group**.

User ID e Superutente

Lo **user ID (UID)** è il numero che identifica in modo **univoco ogni utente** nel sistema UNIX. L'utente con **UID 0** è il **superutente** (root), con privilegi amministrativi completi. Un breve **programma in C** può essere usato per stampare UID e GID (Group ID) correnti utilizzando le funzioni `getuid()` e `getgid()`, dimostrando come tali informazioni siano accessibili a livello di codice.

```
#include "apue.h"
int
main(void)
{
    printf("uid = %d, gid = %d\n", getuid(), getgid());
    exit(0);
}

$ ./a.out
uid = 205, gid = 105
```

E' necessario includere un file header (contenuto nei sorgenti che scaricherete). Esso contiene alcuni header di sistema standard e definisce costanti e prototipi di funzioni.

Logging in

Dopo aver effettuato il **login**, l'utente può impartire **comandi** al sistema tramite un interprete chiamato **shell**. La shell riceve l'input, lo interpreta ed esegue i comandi. Ogni utente dispone di una shell assegnata **di default**. Tra le shell più diffuse troviamo:

- **Bourne shell** (/bin/sh), sviluppata da Steve Bourne;
- **C shell** (/bin/csh), progettata da Bill Joy;
- **Korn shell** (/bin/ksh), creata da David Korn;
- **Bourne Again shell** (/bin/bash), la shell GNU.

La **shell** rappresenta un'interfaccia potente e flessibile tra l'utente e il sistema operativo.

File e Directory

Il **filesystem di UNIX** è strutturato come un insieme di **file** e **directory**, organizzato in modo **gerarchico**. Alla base di questa struttura si trova la **directory radice**, indicata con il simbolo /, da cui si dirama l'intero albero delle directory del sistema.

Ogni **directory** agisce come una **tabella** che associa un **nome** a un **puntatore** verso una struttura dati contenente le informazioni del file o della sottodirectory corrispondente. Questa struttura, che rappresenta un **nodo del filesystem**, conserva vari **attributi** tra cui il **tipo di file**, la sua **dimensione**, l'**utente proprietario**, i **permessi di accesso** e altri metadati fondamentali per il sistema operativo.

Esempio: Elencare i file di una directory (comando ls)

```
#include <sys/types.h>
#include <dirent.h>
#include "apue.h"           // Include header del libro APUE per funzioni di utilità

int main(int argc, char *argv[])
{
    DIR *dp;                // Dichiarare un puntatore a una directory aperta
    struct dirent *dirp;     // Dichiarare un puntatore a una voce (file) della directory

    if (argc != 2)
        err_quit("a single argument (the dir name) is required"); // Termina se non viene passato un nome
    // di directory

    if ((dp = opendir(argv[1])) == NULL) // Apre la directory indicata
        err_sys("can't open %s", argv[1]); // se fallisce, mostra errore di sistema

    while ((dirp = readdir(dp)) != NULL) // Legge la prossima voce nella directory dp e la assegna a dirp.
    // Ogni chiamata successiva legge la voce successiva. Restituisce NULL alla fine
        printf("%s\n", dirp->d_name); // Legge ogni voce della directory e stampa il suo nome

    closedir(dp);            // Chiude la directory e libera risorse

    exit(0);                 // Termina il programma con successo
}
```

Questo programma elenca i nomi di tutti i file e le cartelle contenuti in una directory specificata da riga di comando, comportandosi in modo simile al comando `ls`. Utilizza `opendir()` per aprire la directory, `readdir()` per leggerne le voci una alla volta, e `closedir()` per chiuderla al termine.

Input/Output

In UNIX, ogni volta che un **programma apre o crea un file**, il **kernel** restituisce un **descrittore di file**, ovvero un numero intero che identifica univocamente il file per tutte le operazioni successive di **lettura e scrittura**. Questi **descrittori** sono numeri interi non negativi gestiti all'interno del processo che esegue il programma.

All'avvio di un programma, la **shell** associa automaticamente tre descrittori principali: **0** per lo **standard input**, **1** per lo **standard output** e **2** per lo **standard error**. Normalmente questi sono collegati al **terminale**, ma è possibile **reindirizzarli** verso un file o un altro dispositivo, ad esempio usando comandi come `ls > file.list`.

I/O non bufferizzato

Il sistema UNIX mette a disposizione un insieme di funzioni per la **gestione diretta dei file** senza buffering. Le principali sono: `open`, `read`, `write`, `lseek` e `close`. Queste operazioni lavorano direttamente con i **descrittori di file**, offrendo un controllo più fine e immediato sull'**input/output a basso livello**, rispetto alle funzioni di I/O bufferizzato che operano su flussi.

Esempio: Leggere dallo std input e scrivere sullo std output (I/O NON Bufferizzato)

```
#include "apue.h"           // Include la libreria APUE (per funzioni come err_sys)

#define BUFFSIZE 8192       // Definisce la dimensione del buffer di lettura/scrittura

int main(void) {

    int n;                  // Variabile per memorizzare il numero di byte letti

    char buf[BUFFSIZE];     // Buffer in cui leggere i dati da stdin

    // Ciclo: legge dallo standard input (fino a BUFFSIZE byte per volta)
    while ((n = read(STDIN_FILENO, buf, BUFFSIZE)) > 0) {

        // Scrive i dati letti sullo standard output
        if (write(STDOUT_FILENO, buf, n) != n)

            err_sys("write error"); // Se la scrittura fallisce, stampa errore e termina
    }

    if (n < 0)

        err_sys("read error");     // Se la lettura fallisce, stampa errore e termina

    exit(0);                     // Uscita con successo
}
```

Questo programma legge dati dallo standard input e li scrive sullo standard output, in blocchi di 8192 byte alla volta.

La riga `#define BUFFSIZE 8192` imposta la dimensione massima di ogni blocco da leggere e scrivere. Viene creato un buffer `buf[BUFFSIZE]` che conterrà temporaneamente i dati letti.

La funzione `read(STDIN_FILENO, buf, BUFSIZE)` prova a leggere fino a 8192 byte dalla tastiera o da un file, e memorizza quanti byte ha letto nella variabile `n`. Se `n > 0`, significa che ha letto dei dati, e questi vengono immediatamente scritti su `stdout` con `write(STDOUT_FILENO, buf, n)`.

La scrittura viene verificata confrontando il numero di byte scritti con `n`. Se per qualche motivo `write()` scrive meno byte di quanti ne ha ricevuti, viene chiamata la funzione `err_sys("write error")` per stampare un messaggio di errore e uscire.

Quando `read()` restituisce 0, vuol dire che è stato raggiunto l'end-of-file (EOF), quindi il ciclo si interrompe. Se `read()` restituisce un valore negativo, si è verificato un errore di lettura, e il programma lo segnala con `err_sys("read error")`.

Alla fine, `exit(0)` termina il programma indicando che è andato tutto bene.

Questo codice si comporta come una copia binaria da input a output, molto simile al comando `cat` quando usato per copiare contenuti tra file o da terminale.

Standard I/O

Le **funzioni di I/O standard** offrono un'interfaccia **bufferizzata** alle primitive di **I/O non bufferizzato**, rendendo più semplice ed efficiente la gestione delle operazioni di lettura e scrittura. Queste funzioni sono definite nell'header `<stdio.h>` e permettono di evitare il controllo diretto sulla **dimensione del buffer**, che altrimenti sarebbe a carico del programmatore.

Un grande vantaggio dell'uso dell'I/O standard è la **gestione semplificata delle righe di input**. Ad esempio, la funzione **`fgets`** legge direttamente un'intera **linea di testo**, mentre una funzione come **`read`** richiede di specificare esattamente il numero di **byte** da leggere.

Esempio: Leggere dallo std input e scrivere sullo std output (I/O Bufferizzato)

```
#include <stdio.h>

int main(void) {
    int c;
    while ((c = getc(stdin)) != EOF) {
        if (putc(c, stdout) == EOF) {
            fprintf(stderr, "Errore di output\n");
            return 1;
        }
        if (ferror(stdin)) {
            fprintf(stderr, "Errore di input\n");
            return 1;
        }
    }
    return 0;
}
```

Differenze principali

Aspetto	I/O Standard (<code>getc</code> / <code>putc</code>)	I/O Low-Level (<code>read</code> / <code>write</code>) 
Bufferizzazione	Sì (automatica, gestita da <code>stdio.h</code>)	No (gestione diretta della quantità di dati)
Funzioni	<code>getc()</code> , <code>putc()</code>	<code>read()</code> , <code>write()</code>
Gestione errori	<code>ferror()</code> , controlli su <code>EOF</code>	verifica dei valori di ritorno (<code>< 0</code>)
Comodità	Maggiore (maneggia anche linee, <code>EOF</code> , buffer)	Più complesso, ma preciso e più performante in certi casi
Contesto d'uso consigliato	Applicazioni generiche, testi, flussi interattivi	File binari, alta efficienza, operazioni su socket

Programmi e Processi

In UNIX, un **programma** è un **file eseguibile** memorizzato nel **filesystem**. Quando viene eseguito, il **kernel** lo carica in **memoria** e ne avvia l'esecuzione utilizzando una delle funzioni della famiglia **exec**.

Nel momento in cui il programma è in esecuzione, prende il nome di **processo**. Ogni processo attivo nel sistema è identificato in modo **univoco** tramite un numero intero chiamato **PID (Process ID)**, che è sempre **non negativo**.

Esempio

```
#include "apue.h"           // Include l'header APUE che fornisce funzioni e costanti di utilità

int main(void) {

    // Stampa un messaggio che include l'ID del processo corrente
    // getpid() restituisce l'identificativo univoco (PID) del processo in esecuzione
    printf("hello world from process ID %d\n", getpid());

    // Termina il programma con codice di uscita 0 (successo)
    exit(0);
}
```

Ogni volta che viene eseguito il programma, il sistema operativo crea un **nuovo processo**, e quindi assegna un **PID diverso**. Questo codice dimostra come si possa **ottenere e stampare il PID** con `getpid()`, utile per il **debug**, il **monitoraggio**, o la **gestione dei processi**.

Controllo dei processi

Per la gestione dell'esecuzione dei **processi** in UNIX, esistono tre funzioni fondamentali:

- **`fork()`**, che serve per creare un nuovo processo duplicando quello esistente;
- **`exec()`**, che sostituisce il contenuto del processo corrente con un nuovo programma;
- **`waitpid()`**, utilizzata per **attendere la terminazione** di un processo figlio e recuperare informazioni sul suo stato di uscita.

Esempio programma che legge i comandi dallo standard input e li esegue

```
#include <sys/types.h>
#include <sys/wait.h>
#include "apue.h"

int main(void) {
    char buf[MAXLINE];    // buffer per leggere il comando da tastiera
    pid_t pid;            // per memorizzare il PID del processo figlio
    int status;           // per salvare lo stato di uscita del figlio

    printf("%% ");        // stampa il prompt: %

    while (fgets(buf, MAXLINE, stdin) != NULL) {
        buf[strlen(buf) - 1] = 0;    // rimuove il carattere '\n' alla fine dell'input

        if ((pid = fork()) < 0)
            err_sys("fork error");    // errore nella creazione del processo figlio

        else if (pid == 0) {
            execlp(buf, buf, (char *)0);    // il figlio esegue il comando inserito
            err_ret("couldn't execute: %s", buf);    // se execlp fallisce, stampa errore
            exit(127);    // esce con codice errore
        }

        if ((pid = waitpid(pid, &status, 0)) < 0)
            err_sys("waitpid error");    // attende la terminazione del figlio

        printf("%% ");    // stampa il prompt di nuovo
    }

    exit(0);
}
```

Questo programma è una **mini-shell** scritta in C che simula, in modo semplificato, il comportamento di un terminale Unix come `bash`. Il suo scopo è leggere un comando da tastiera, eseguirlo tramite un processo figlio e poi tornare ad attendere un nuovo comando.

All'inizio, viene stampato a video il prompt `%`, per indicare all'utente che può inserire un comando. Il programma poi entra in un ciclo `while` che continua a leggere righe dalla tastiera finché l'utente non chiude lo standard input (ad esempio premendo `Ctrl+D`).

La lettura del comando avviene con `fgets()`, che salva l'intera riga in un buffer chiamato **buf**. Per rimuovere il carattere di newline `\n` alla fine dell'input, si sostituisce l'ultimo carattere con `\0`, rendendo la stringa compatibile con le funzioni di esecuzione dei comandi.

Subito dopo, il programma crea un **processo figlio** usando la funzione `fork()`. Se la `fork()` fallisce, stampa un errore e termina. Se invece siamo nel processo figlio (`pid == 0`), il figlio prova a eseguire il comando inserito usando `execlp()`. Questa funzione cerca il nome del comando nel **PATH** di sistema, e se lo trova, rimpiazza completamente l'immagine del processo corrente con il nuovo programma. In altre parole, **il figlio diventa il programma eseguito** (ad esempio `ls`, `date`, `who`, ecc.). La riga `execlp(buf, buf, (char *)0);` significa: “cerca il comando con nome `buf` e esegilo passando `buf` come `argv[0]`, senza ulteriori argomenti”. Se l'esecuzione fallisce (ad esempio perché il comando non esiste), viene stampato un messaggio di errore e il processo figlio termina con `exit(127)`.

Nel frattempo, il **processo padre** chiama `waitpid()` per aspettare che il figlio finisca. Solo dopo che il figlio è terminato, il padre torna a stampare il prompt `%` e il ciclo ricomincia, pronto per un nuovo comando.

Il ciclo si interrompe quando l'utente invia **EOF** (fine input), e il programma esce con **exit(0)**, segnalando un termine corretto.

Questo programma legge un comando, crea un figlio che prova a eseguirlo con **execvp**, il padre attende che finisca, e poi ripropone il prompt. È una base perfetta per capire il cuore del funzionamento di una shell Unix.

Gestione degli errori

Nel sistema UNIX, le funzioni che segnalano un **errore** restituiscono spesso un **valore negativo**. Quando ciò avviene, viene impostata una variabile globale chiamata `errno`, che contiene un **codice numerico** associato al tipo di errore avvenuto.

Questa variabile è dichiarata come `extern int errno;` e la sua definizione è accessibile tramite l'header `<errno.h>`, dove sono elencati tutti i possibili codici e le relative **costanti simboliche** (es. `ENOENT`, `EACCES`, ecc.).

```
#include <string.h>

char *strerror(int errnum);
```

Per ottenere una **descrizione testuale** dell'errore, il linguaggio C fornisce la funzione `strerror(int errnum)`, dichiarata in `<string.h>`, che restituisce una **stringa** contenente il messaggio di errore corrispondente al valore di `errnum`.

```
#include <stdio.h>

void perror(const char *msg);
```

In alternativa, è possibile utilizzare la funzione `perror(const char *msg)`, inclusa in `<stdio.h>`, che stampa il **messaggio passato come argomento**, seguito dal messaggio di errore associato al valore corrente di `errno`. Il tutto viene visualizzato con **formattazione automatica**, utile per il debug o la diagnostica dei problemi a runtime.

Esempio

```
#include <errno.h>

#include <stdio.h>

#include <string.h>

int main(int argc, char *argv[]) {

    fprintf(stderr, "EACCES: %s\n", strerror(EACCES));

    errno = ENOENT;

    perror(argv[0]);

    exit(0);

}
```

Questo programma mostra come stampare messaggi di errore in C utilizzando **strerror** e **perror**. All'inizio stampa su **stderr** la descrizione dell'errore **EACCES**, che rappresenta "Permission denied". Poi imposta manualmente la variabile globale **errno** al valore **ENOENT**, che corrisponde all'errore "No such file or directory". Infine, usa **perror** per stampare un messaggio

di errore completo, composto dal nome del programma (`argv[0]`) seguito dalla descrizione dell'errore presente in `errno`. Il programma termina con `exit(0)`.

La prima riga (`EACCES: Permission denied`) viene da `fprintf + strerror`, ed è completamente **manuale**.

La seconda riga (`./a.out: No such file or directory`) viene da `perror(argv[0])`, che:

- stampa il nome passato (`argv[0]`, cioè `./a.out`)
- aggiunge i due punti :
- poi stampa la descrizione testuale dell'errore memorizzato in `errno`

Segnali

In UNIX, i **segnali** rappresentano un meccanismo per **notificare a un processo** che si è verificata una determinata **condizione**. Questi segnali vengono inviati dal sistema operativo o da altri processi, per segnalare eventi asincroni.

Un esempio comune è la pressione di **Ctrl+C** da tastiera, che invia al processo attivo il segnale `SIGINT`, richiedendone l'interruzione.

Quando un processo **riceve un segnale**, può gestirlo in tre modi: può **ignorarlo**, **accettare l'azione predefinita** (come la terminazione), oppure **definire una funzione personalizzata** da eseguire in risposta a quel segnale. Questa possibilità rende i segnali uno strumento molto potente per **controllare il comportamento** dei processi in condizioni particolari.

Chiamate di sistema

Tutti i sistemi operativi, incluso UNIX, mettono a disposizione dei **punti di accesso al kernel** chiamati **chiamate di sistema**. Queste rappresentano l'interfaccia attraverso cui i **programmi utente** possono **richiedere servizi** fondamentali, come la **creazione di file**, la **gestione dei processi** o la **comunicazione tra processi**.

Funzioni di libreria e kernel

Le **chiamate di sistema** sono implementate nel **kernel** e ogni processo che ne richiede l'esecuzione **trasferisce il controllo al sistema operativo**. Tuttavia, i programmatori interagiscono con esse attraverso **funzioni di libreria C**, che offrono una **interfaccia più semplice** e familiare, ad esempio `open(path, mode)`.

Questo strato di libreria traduce la richiesta in una chiamata vera e propria, salvando i **parametri nei registri** e generando un'istruzione di tipo **trap**, che consente il passaggio dal **modo utente** al **modo kernel**.

Le **funzioni di libreria** in C forniscono **servizi di utilità generale** pensati per facilitare il lavoro del **programmatore**. A differenza delle **system call**, non rappresentano un punto di ingresso diretto al **kernel**, anche se alcune di esse possono **invocare chiamate di sistema** per svolgere determinati compiti.

Ad esempio, la funzione `printf` può internamente utilizzare la system call `write()` per stampare su schermo, mentre funzioni come `strcpy` (che copia stringhe) o `atoi` (che converte da ASCII a intero) lavorano solo in **user space** e **non coinvolgono il sistema operativo**.

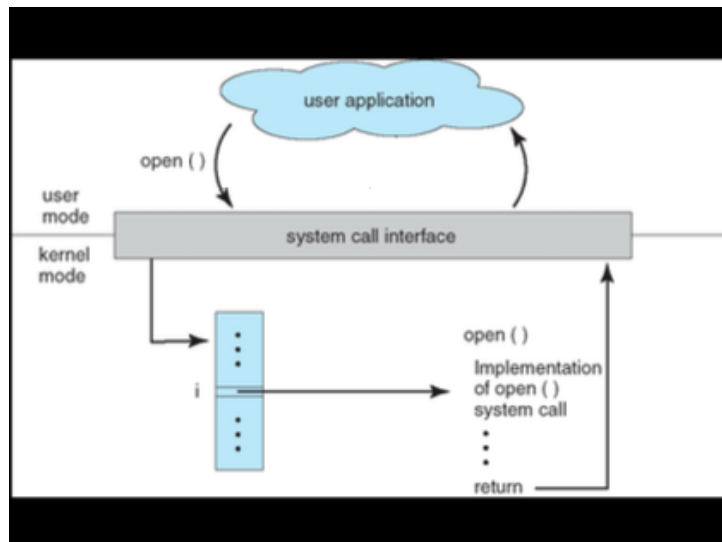
Una caratteristica importante è che le funzioni di libreria possono essere **sostituite** da altre che svolgono lo stesso compito, cosa che generalmente **non è possibile** con le **system call**, che sono vincolate all'interfaccia fornita dal kernel.

Queste funzioni sono documentate nella **sezione 3 del manuale UNIX** (man 3), che raccoglie tutte le **funzioni di libreria standard**.

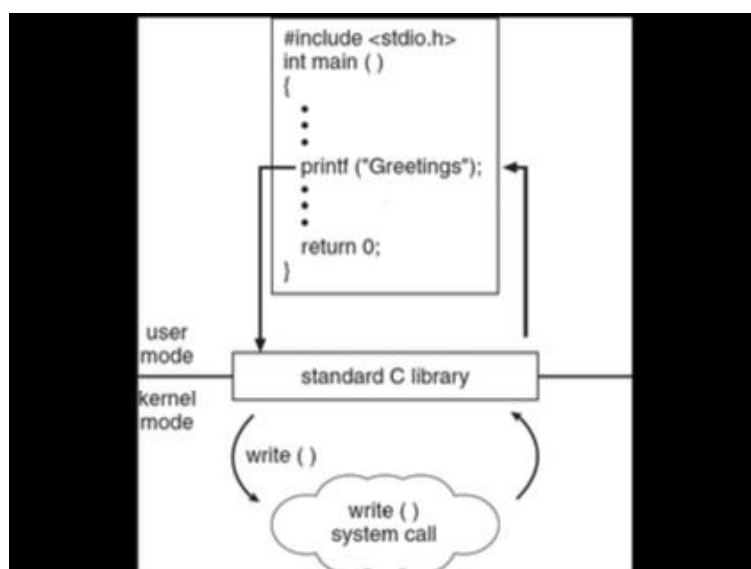
Meccanismo tecnico

Quando una funzione come `open()` viene invocata, il sistema:

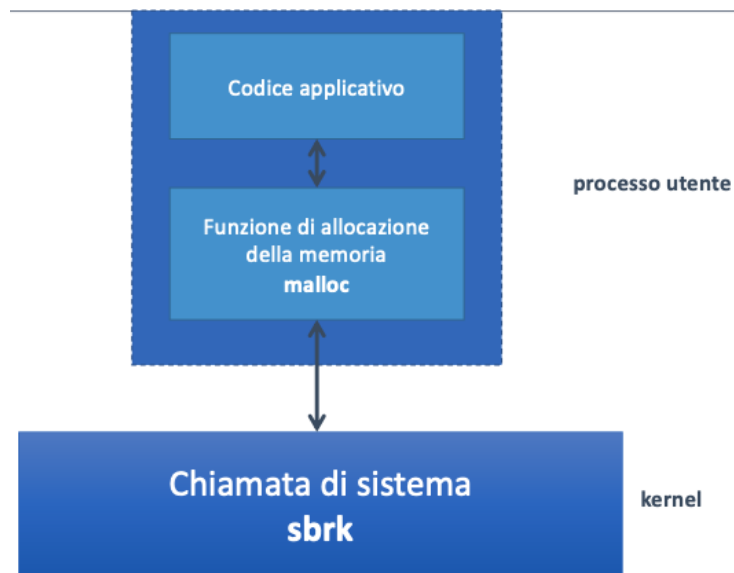
1. salva gli **argomenti** es. ("`file.txt`", `O_RDONLY`) e l'**identificatore della chiamata di sistema** (un numero costante che identifica la system call),
2. esegue una **trap** (interrupt software) che commuta il processo in **modalità kernel**,
3. esegue il servizio richiesto e lo elabora: in questo caso apre il file, controlla i permessi, riserva strutture interne ecc..
4. Dopo aver eseguito l'operazione si ritorna al programma utente con un valore di **successo** o **errore**.



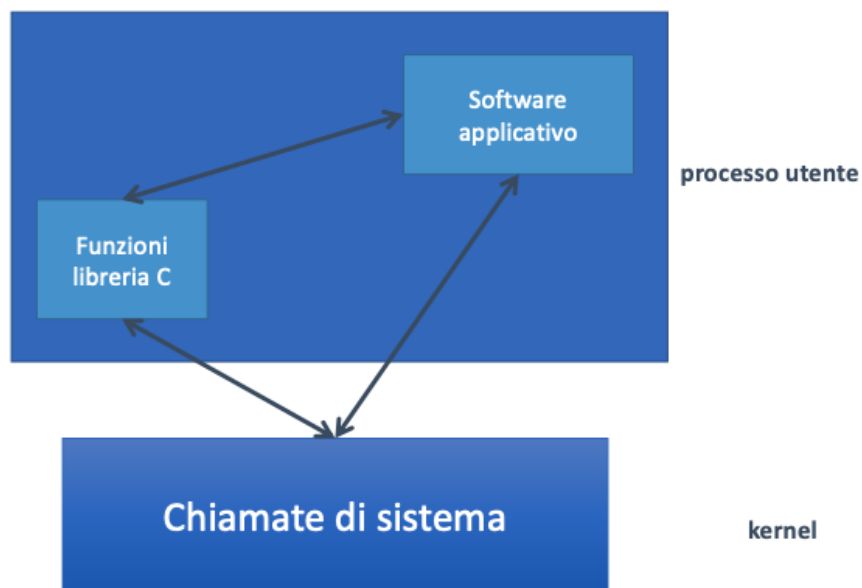
Lo schema presentato mostra il flusso da un'applicazione utente, che invoca la funzione `open()`, attraverso l'**interfaccia delle system call**, fino alla vera e propria **implementazione della chiamata nel kernel**, evidenziando la separazione tra **user mode** e **kernel mode**.



Nel primo schema viene mostrato come una funzione della **libreria standard C**, come `printf()`, operi in **user mode** ma internamente invochi la **system call** `write()`. In questo caso, il programmatore utilizza `printf`, che fa parte della libreria `stdio.h`, senza doversi preoccupare del dettaglio che la vera operazione di scrittura viene eseguita dal **kernel** tramite `write()`. Questo rappresenta il tipico **livello di astrazione** offerto dalle librerie C.



Il secondo schema evidenzia come una funzione **di allocazione dinamica della memoria**, come `malloc`, sia realizzata a sua volta sfruttando una **chiamata di sistema sottostante**, come `sbrk`. Il **codice applicativo** (utente) chiama `malloc`, che in alcuni casi può richiedere ulteriore memoria al sistema tramite `sbrk`, entrando così in **modalità kernel**. Anche qui, l'astrazione della libreria C consente all'utente di lavorare a un livello più alto, senza interagire direttamente con `sbrk`.



Nel terzo schema viene mostrata in generale la **struttura delle interazioni** in UNIX tra un **programma utente**, le **funzioni di libreria C** e le **chiamate di sistema**. Il software applicativo può invocare direttamente una **funzione di libreria**, oppure una chiamata di sistema, e le librerie stesse possono essere implementate ricorrendo alle system call. È un'architettura modulare che separa nettamente il **processo utente** (user space) dal **kernel** (kernel space), aumentando sicurezza e stabilità.

Capitolo 2 – Programmazione in C in ambiente Unix/Linux

Nel contesto dei **sistemi operativi**, il linguaggio **C** è lo strumento ideale per accedere alle **system call** di UNIX, ovvero quelle funzioni che permettono di interagire direttamente con il **kernel** per gestire file, eseguire più processi (multitasking), e realizzare comunicazioni tra processi (**IPC**).

La scelta del linguaggio di programmazione è discussa da anni, ma nel campo della **programmazione di sistema**, il **linguaggio C** è quasi universalmente adottato. Questo perché:

- è **vicino all'hardware** e produce codice facilmente traducibile in linguaggio macchina;
- consente **accesso diretto alla memoria** attraverso puntatori;
- i tipi di dato sono modellati sulle strutture dell'hardware;
- UNIX stesso è **storicamente scritto in C**, così come il kernel di **Linux**.

Di conseguenza, moltissimi **driver** e **strumenti di sistema** sono scritti in C per efficienza e portabilità.

Compilazione in ambiente Unix/Linux

Per compilare codice C, ogni sistema Unix fornisce un compilatore standard, chiamato **cc** o, in ambiente GNU/Linux, **gcc**. Questo è conforme allo **standard POSIX** e permette di compilare file sorgente (`.c`) in file oggetto (`.o`) o eseguibili.

La sintassi generale è:

```
gcc <opzioni> <file>
```

Ad esempio, per compilare `source.c` e produrre l'eseguibile predefinito `a.out`:

```
gcc source.c
```

Oppure per specificare il nome dell'output:

```
gcc source.c -o nome_output
```

Opzioni comuni del compilatore

Il compilatore `gcc` mette a disposizione varie opzioni:

- **-c** compila ma non collega (genera solo il file oggetto),
- **-g** aggiunge informazioni di debug per strumenti come **gdb**,
- **-o file** definisce il nome dell'output,
- **-l nome** collega una libreria specifica (`libnome.a` o `libnome.so`),
- **-v** fornisce dettagli sull'intera fase di compilazione.

Compilazione e costruzione di un programma

Una volta scritto, un programma deve essere **trasformato in un eseguibile**. Questo processo avviene attraverso una **sequenza di fasi**, in cui il codice sorgente viene convertito progressivamente in **forme intermedie**: **preprocessato**, **assembly**, **codice oggetto**, fino ad arrivare al **collegamento con librerie** per ottenere il file finale eseguibile.

Comprendere ogni fase è fondamentale per ottimizzare tempo e risorse. Il **preprocessing** si occupa della sostituzione di macro e direttive, la **fase assembly** genera codice leggibile dal processore, mentre il **linking** permette di riutilizzare codice già esistente, combinando diversi moduli in un unico eseguibile.

Le **forme intermedie** possono anche essere salvate per evitare di ricompilare tutto il progetto in caso di modifiche parziali, migliorando i tempi di **build successivi**.

Codice oggetto e linking

I passaggi cruciali nella costruzione di un eseguibile sono la **compilazione** e il **linking**. La compilazione trasforma il **codice sorgente**, scritto in C, in un **codice oggetto**, cioè istruzioni in linguaggio macchina che il processore può comprendere. Tuttavia, questo codice oggetto **non è eseguibile da solo**: deve essere **collegato (linked)** con altri moduli per diventare un programma funzionante.

Ad esempio, se si scrive una funzione come `OurSquareRoot()` in un file `sqrt.c`, essa può essere compilata usando `gcc -c sqrt.c`, ottenendo un file oggetto `sqrt.o`. Questo file sarà poi **collegato con altri**, come `main.o`, per creare il programma completo.

Entrambi i file (`main.c` e `sqrt.c`) vengono compilati separatamente con l'opzione `-c`, producendo due file oggetto: **`main.o` e `sqrt.o`**. Questi vengono infine collegati insieme con un comando come `gcc main.o sqrt.o`, che genera l'eseguibile `a.out`.

Eseguendo `./a.out`, l'utente può inserire un numero e ottenere come risultato la radice quadrata calcolata dalla funzione `OurSquareRoot()`.

Compilando invece solo `sqrt.o`, senza includere `main.o`, si otterrà un errore come **"undefined reference to main"**, perché il compilatore non trova il punto d'ingresso del programma.

Un eseguibile deve sempre contenere **esattamente una funzione `main()`**, poiché essa indica **dove inizia l'esecuzione**.

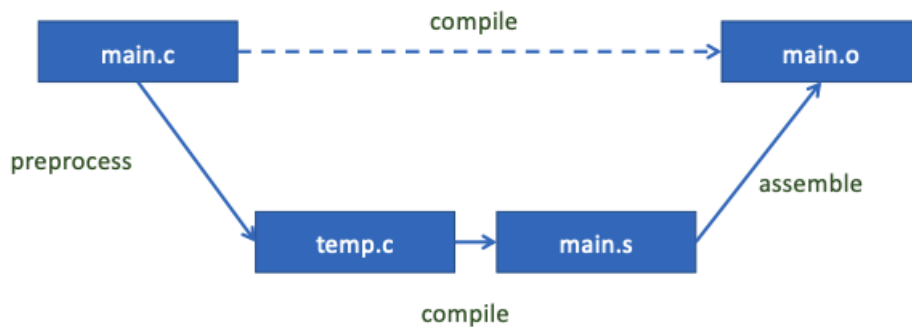
Compilazione e Linking

Durante la **compilazione**, il comando `gcc`, a meno che non venga indicato diversamente, esegue anche il **linking automatico**, eliminando così eventuali file oggetto intermedi. Questo approccio semplifica il lavoro del programmatore, evitando di dover eseguire manualmente ogni fase. Tuttavia, quando si lavora con **più file sorgenti**, è preferibile compilare ogni file separatamente in **codice oggetto**, in modo da ricompilare solo i file modificati. Questa tecnica migliora l'efficienza, soprattutto in progetti con molti file sorgenti.

Il **linking** serve a **combinare file oggetto** in un unico file eseguibile. Può includere anche codice oggetto proveniente da **librerie**, che contengono funzioni di uso frequente. Ad esempio, la libreria standard del C `libc.a` fornisce il supporto per funzioni come `printf()` e `strcmp()`, ed è spesso già disponibile nel sistema. Poiché queste funzioni sono usate molto frequentemente, non devono essere ricompilate ogni volta.

Quando si specifica un'opzione come `-lc`, si richiede a `gcc` di **linkare esplicitamente** la libreria `libc.a`. Di norma, `gcc` fa questo automaticamente. L'opzione `-L` serve a specificare **percorsi aggiuntivi** in cui cercare librerie. I file oggetto delle librerie sono solitamente **copiati nell'eseguibile**, con il rischio di ridondanza se più eseguibili usano gli stessi oggetti.

Nel caso del **linking dinamico**, il **codice oggetto della libreria non viene copiato** nell'eseguibile. Viene invece caricato a runtime, solo se necessario. Questo comporta un minore utilizzo di spazio, ma rende l'eseguibile dipendente dalla presenza della libreria nel sistema. Con il comando `-static`, si ottiene invece un **linking statico**, che include tutto il codice nell'eseguibile, rendendolo più grande ma autonomo.



Il processo di compilazione si compone di **tre fasi principali: preprocessing, compiling e assembling**. Durante il preprocessing, si espandono macro e direttive; con il compiling si genera codice assembly; infine, l'assembling trasforma l'assembly in codice oggetto (`main.o`).

Debug

Il **debugger** è un programma che consente di controllare l'esecuzione di un altro programma, aiutando a trovare e correggere errori. Un esempio di codice C semplice (`sum.c`) mostra una somma condizionata con un ciclo `for`.

gdb è il debugger standard in ambiente GNU/Linux. Dopo aver compilato un programma con l'opzione **-g** (per includere informazioni di debug), è possibile usare comandi come:

- **run** per eseguire il programma,
- **break** per inserire un punto di interruzione,
- **step**, **next**, **continue** per controllare l'esecuzione riga per riga,
- **print** e **display** per vedere i valori delle variabili,
- **list** e **where** per ispezionare il codice o la posizione nel programma.

Durante la compilazione con **-g**, viene generata anche la **tabella dei simboli**, che include nomi di variabili e altre informazioni utili per il debug.

Infine, si può osservare il comportamento del debugger nel tempo, grazie ai comandi **display** (visualizzazione automatica di variabili) e **clear** (per rimuovere breakpoint).

Organizzazione del codice

Nel linguaggio **C**, un programma può essere scritto interamente in un singolo **file sorgente** o suddiviso in più **moduli**. Inizialmente, per semplicità, si tende a usare un solo file **.c**, dove sono contenute tutte le **funzioni** necessarie, inclusa la **main()** e altre funzioni come **reverse()** per invertire una stringa. Questo approccio va bene per programmi molto semplici, ma presenta diversi **limiti**. Infatti, ogni piccola modifica richiede la **ricompilazione** dell'intero file, con conseguente aumento dei **tempi di compilazione**. Inoltre, le funzioni definite all'interno del file non sono facilmente **riutilizzabili** in altri progetti. Una possibile ma pessima soluzione è quella di copiare il codice manualmente da un progetto all'altro: questa tecnica, chiamata **"cut & paste"**, complica la **manutenzione** e porta a inefficienze, perché ogni copia occupa spazio e deve essere aggiornata separatamente.

Per superare questi limiti, si adotta una **suddivisione in più moduli**. In questo schema, un programma è diviso in più file **.c**, ognuno dei quali può essere **compilato indipendentemente** e poi **collegato** agli altri in un unico **eseguibile**. Questo consente di **ricompilare solo i file modificati**, risparmiando tempo, e di **riutilizzare** lo stesso file in più progetti. Quando un modulo contiene una funzione usata altrove, è necessario dichiarare il suo **prototipo** in ogni file che la richiama, ad esempio con **void reverse(char *, char *)**; . Inoltre, anche le **direttive del pre-processore** e le **definizioni di tipo** devono essere duplicate o, preferibilmente, **centralizzate**.

Per facilitare questo, si introduce un **file di intestazione .h (header file)**, come ad esempio **modulo.h**, dove si inseriscono le **dichiarazioni delle funzioni** e le **definizioni comuni**. Il modulo corrispondente, **modulo.c**, includerà questo **header**, così come ogni altro file che intende utilizzare quel modulo. In pratica, si scrive **#include "modulo.h"** sia nel file stesso che in quelli che lo impiegano. Ogni file **.c** può essere poi compilato separatamente con **gcc -c modulo.c**, producendo un **file oggetto**

.o che contiene **codice macchina** e la **tabella dei simboli**. Questa tabella permette, tramite il **compilatore gcc** o il **loader ld**, di combinare il modulo con altri file oggetto e ottenere un **programma eseguibile** completo.

Questa **organizzazione modulare** rende il codice più **manutenibile**, **riutilizzabile** e **facile da aggiornare**, risolvendo i principali problemi associati ai programmi scritti in un **singolo file**.

Compilazione separata

Per ottenere una **compilazione separata**, si usa l'opzione **-c** con **gcc**, ad esempio:

```
$ gcc -c reverse.c
$ gcc -c usaRev.c
```

Questi comandi generano i file oggetto **reverse.o** e **usaRev.o**, mantenendo intatti i file sorgente originali. Con il comando **ls -l** si può verificare la presenza di questi file.

In alternativa, è possibile compilare entrambi i file sorgenti in un solo comando:

```
$ gcc -c reverse.c usaRev.c
```

Successivamente, si esegue il **linking** dei file oggetto con:

```
$ gcc reverse.o usaRev.o -o usaRev
```

Questo crea l'eseguibile chiamato **usaRev**.

Un vantaggio della compilazione separata è che se si modifica solo **reverse.c**, sarà sufficiente ricompilare quel file:

```
$ gcc -c reverse.c
```

...e poi ripetere solo il linking:

```
$ gcc reverse.o usaRev.o -o usaRev
```

Questo approccio permette di risparmiare tempo e risorse, evitando la ricompilazione dell'intero progetto.

MakeFile

Il **makefile** è uno strumento fondamentale per la gestione automatica della **compilazione**, **linking** e **installazione** di programmi suddivisi in più file sorgente. In un progetto complesso, dove i file sorgente e i file oggetto sono numerosi e soggetti a modifiche frequenti, ricompilare manualmente ogni file modificato e ricostruire l'eseguibile può diventare un'operazione lunga e ripetitiva. Per questo motivo Unix mette a disposizione l'utility **make**, che consente di automatizzare la manutenzione e l'aggiornamento dei programmi.

Quando si modifica frequentemente il codice, è necessario controllare se qualche file sorgente è cambiato, ricompilare solo i file modificati e infine ricostruire l'eseguibile. Tutte queste sono **operazioni ripetitive** che il comando **make** automatizza, compilando i file necessari e aggiornando l'eseguibile solo quando necessario.

Funzionamento informale di make

Il comando **make** verifica se i **file sorgente** sono stati modificati rispetto ai relativi **file oggetto**, compila i file modificati e ricostruisce il programma eseguibile aggiornato. Questo processo si basa sulle **regole di dipendenza** contenute in un file chiamato **makefile**, che specifica quali file devono essere compilati e come devono essere collegati tra loro.

Per impostazione predefinita, **make** cerca un file chiamato **Makefile** o **makefile** nella directory corrente. È comunque possibile specificare un file diverso con l'opzione **-f**, ad esempio:

```
make -f nomefile
```

Struttura di un Makefile

Un **makefile** è composto da una serie di **regole**. Ogni regola ha la seguente struttura:

```
targetList: dependencyList
[TAB] commandList
```

Il **targetList** rappresenta il file da generare, il **dependencyList** elenca i file da cui dipende, mentre il **commandList** contiene i comandi da eseguire per creare il target. Ogni comando deve iniziare con un **TAB**, non con spazi.

Ad esempio:

```
prog: main.o mod1.o
    gcc -o prog main.o mod1.o
```

All'interno del file è possibile inserire **commenti** preceduti da #.

Esempio semplice di Makefile

```
# makefile per prog
prog: main.o prog1.o prog2.o prog3.o prog4.o
    gcc -o prog main.o prog1.o prog2.o prog3.o prog4.o

main.o: main.c
    gcc -c main.c

prog1.o: prog1.c
    gcc -c prog1.c

prog4.o: prog4.c
    gcc -c prog4.c
```

Regole del makefile

Una **linea di dipendenza** definisce la relazione tra un **target** (a sinistra del :) e uno o più **file di dipendenza** (a destra).

Una **linea di comando** specifica le azioni che **make** deve eseguire per generare il target a partire dalle dipendenze.

Se i file di dipendenza sono più recenti del target, oppure se il target non esiste, la linea di comando viene eseguita.

Costruzione dell'albero delle dipendenze

make costruisce un **albero delle dipendenze** partendo dai **target** e navigando tra le **dependencyList**. Visita l'albero in **profondità**, eseguendo i comandi solo se un file sorgente è più recente del relativo file oggetto o se un file oggetto non esiste.

Regole predefinite

Non è necessario specificare sempre tutte le regole e dipendenze. **make** è in grado di riconoscere automaticamente file con estensione **.c** come file sorgente C, applicando regole predefinite per la compilazione.

Esempio con regole predefinite:

```
prog: main.o prog1.o prog2.o
```

```
gcc -o prog main.o prog1.o prog2.o
```

In questo caso non sono necessarie le regole per compilare i singoli `.c` in `.o`, perché `make` sa già come farlo.

Makefile parametrico

Per evitare la ripetizione di nomi di file in più punti, si possono usare le **macro**, che assegnano un nome a una stringa, da riutilizzare più volte:

```
OGGETTI = main.o prog1.o

prog: $(OGGETTI)
    gcc -o prog $(OGGETTI)
```

Le **macro** sono definite nella forma:

```
nome = stringa
```

e richiamate come `$(nome)` oppure `${nome}`.

Un esempio completo con più macro:

```
OGGETTI = qdrig.o pippo.o
OGGETTI1 = qdcol.o pluto.o
MAIN = driver.o
MAIN1 = driver_uno.o

rows: $(OGGETTI) $(MAIN)
    gcc -o rows $(OGGETTI) $(MAIN)

columns: $(OGGETTI1) $(MAIN1)
    gcc -o columns $(OGGETTI1) $(MAIN1)
```

Esempio pratico: usaPal

Un esempio di makefile per il programma `usaPal`, che utilizza i moduli `palindroma` e `reverse`, è il seguente:

```
usaPal: usaPal.o palindroma.o reverse.o
    gcc usaPal.o palindroma.o reverse.o -o usaPal

usaPal.o: usaPal.c palindroma.h
    gcc -c usaPal.c

palindroma.o: palindroma.c palindroma.h reverse.h
    gcc -c palindroma.c

reverse.o: reverse.c reverse.h
    gcc -c reverse.c
```

Spiegazione pratica del contenuto:

```
usaPal: usaPal.o palindroma.o reverse.o
    gcc usaPal.o palindroma.o reverse.o -o usaPal
```

Questa è la **regola principale**. Dice:

"Per creare l'eseguibile `usaPal`, ho bisogno dei file oggetto `usaPal.o`, `palindroma.o` e `reverse.o`."

La riga sotto è il **comando** che `make` esegue per generare l'eseguibile:
Compila e **linka** i file oggetto con `gcc` per produrre il file eseguibile `usaPal`.

```
usaPal.o: usaPal.c palindroma.h
gcc -c usaPal.c
```

Qui stai dicendo:

"Per ottenere `usaPal.o`, serve il file sorgente `usaPal.c` e l'header `palindroma.h`."

Il comando `gcc -c usaPal.c` compila il **solo file sorgente in codice oggetto**, senza eseguibile finale.

```
palindroma.o: palindroma.c palindroma.h reverse.h
gcc -c palindroma.c
```

Simile al precedente:

"Per creare `palindroma.o`, ho bisogno del file `palindroma.c` e degli header `palindroma.h` e `reverse.h`."

Compila `palindroma.c` in `palindroma.o`.

```
reverse.o: reverse.c reverse.h
gcc -c reverse.c
```

"Per ottenere `reverse.o`, uso `reverse.c` e `reverse.h`."

Compila `reverse.c` in `reverse.o`.

Versione semplificata usando regole predefinite:

```
usaPal: usaPal.o palindroma.o reverse.o
gcc usaPal.o palindroma.o reverse.o -o usaPal
```

```
usaPal.o: palindroma.h
palindroma.o: palindroma.h reverse.h
reverse.o: reverse.h
```

Makefile tipico

Un **makefile standard** definisce target come **all**, **install**, e **clean**.

- **all** è il target predefinito, compila l'intero programma.
- **install** copia i binari nella directory di destinazione.
- **clean** elimina file oggetto ed eseguibili per ripulire la directory.

Esempio:

```
prefix=/usr/local
bindir=${prefix}/bin
```

```
all: usaPal usaRev
```

```
install:
    cp usaPal usaRev $(bindir)
```

```
clean:
    rm *.o usaPal usaRev
```

```
usaPal: palindroma.o reverse.o
usaPal.o: palindroma.h
palindroma.o: palindroma.h reverse.h
```

```
usaRev: reverse.o
usaRev.o: reverse.h
reverse.o: reverse.h
```

Quindi, l'uso di **make** e dei **makefile** permette di strutturare la compilazione di progetti C in modo efficiente, modulare e facilmente manutenibile. Le **macro**, le **regole di dipendenza**, e i **target tipici** come **all**, **install** e **clean**, sono strumenti fondamentali per automatizzare processi altrimenti complessi e ripetitivi.

Compressione dei File

La **compressione dei file** e la **gestione degli archivi** in ambiente Unix/Linux è strettamente legata all'uso del comando **tar**, che consente di raccogliere più file e directory in un unico file chiamato archivio. Questo tipo di file non è compresso di per sé, ma è utile per organizzare e trasferire grandi quantità di dati.

Il comando **tar** (acronimo di **tape archive**) è stato originariamente creato per salvare dati su nastri magnetici, ma oggi viene utilizzato per creare archivi sia su dispositivi che su file. Utilizzando l'opzione **f**, si specifica il nome dell'output (il file dell'archivio), mentre l'estensione tipica di un archivio creato è **.tar**. Se si indica una directory, vengono archiviate anche tutte le sue sottodirectory.

Un archivio può essere creato con l'opzione **c**, che serve per la creazione, combinata con **f** che specifica il file:

```
tar cvf myarch.tar mydir
```

Questo comando archivia la directory `mydir` (con tutte le sue sottodirectory) nel file `myarch.tar`.

Per estrarre il contenuto si usa l'opzione **x**:

```
tar xvf myarch.tar
```

L'opzione **v** è opzionale e serve solo a visualizzare i file estratti.

Sebbene **tar** gestisca l'archiviazione, per la **compressione** è comune associarlo a strumenti esterni come **gzip**, **bzip2** o (storicamente) **compress**. Questi strumenti non fanno parte di tar, ma possono essere utilizzati in sequenza o integrati nei comandi GNU con le opzioni aggiuntive:

- **z** per gzip
- **j** per bzip2
- **Z** per compress

Ad esempio, per archiviare e comprimere in due passaggi:

```
tar cf nome_tarball.tar file1 file2
gzip nome_tarball.tar
```

Il risultato sarà `nome_tarball.tar.gz`.

Per visualizzare il contenuto di un archivio compresso:

```
tar tvf nome_tarball.tar.gz
```

Per estrarre i file, si può usare:

```
tar xf nome_tarball.tar.gz
```

Oppure, in due passaggi:

```
gunzip nome_tarball.tar.gz
tar xf nome_tarball.tar
```

Nel caso si voglia usare **bzip2** al posto di **gzip**, si sostituiscono i comandi gzip/gunzip con bzip2/bunzip2, e le estensioni .gz con .bz2.

Con il supporto delle opzioni GNU, è possibile fare tutto in un solo passaggio:

```
tar czf nome.tar.gz file1 file2      # usa gzip
tar cjf nome.tbz2 file1 file2        # usa bzip2
tar cZf nome.tar.Z file1             # usa compress
```

La decompressione segue una sintassi simile, con l'opzione x combinata a z, j o Z:

```
tar xzf nome.tar.gz      # gzip
tar xjf nome.tar.bz2     # bzip2
tar xZf nome.tar.Z       # compress
```

Capitolo 3 – Introduzione alla shell UNIX

Definizione e Funzionalità di una Shell

La **shell** è un **programma speciale** che funge da **interfaccia** tra l'utente e il **kernel** del sistema UNIX/Linux. Mentre il **kernel** viene caricato in memoria al momento del **boot** e gestisce costantemente il sistema (memoria, processi, file, comunicazione), la **shell** e gli altri programmi risiedono inizialmente su disco e vengono avviati solo su richiesta. Una volta eseguiti, il **kernel** li carica, li gestisce e li rimuove dalla memoria quando hanno terminato. La **shell**, in particolare, è un **programma di utilità** che si avvia dopo il login dell'utente e consente l'interazione col sistema, interpretando i **comandi digitati** o contenuti in **script**.

Dopo il login, la **shell** si mette in attesa dell'**input** dell'utente. Quando l'utente immette un **comando**, è la shell ad occuparsi del suo **parsing**, della gestione dei **caratteri jolly** (wildcard), della **redirezione**, delle **pipe** e del **controllo dei job**. Se il comando viene trovato, viene eseguito. Durante l'apprendimento di UNIX/Linux, si usa spesso la shell in modo **interattivo**, cioè scrivendo direttamente comandi dal **prompt**. Tuttavia, quando un insieme di comandi viene usato spesso, è utile **automatizzarlo** creando degli **script**, ovvero file contenenti comandi che vengono eseguiti sequenzialmente. Questi script possono contenere anche **decisioni logiche**, **cicli**, **test** e **gestione dei file**, rendendoli strumenti molto potenti.

Le Shell di Unix

Nel mondo **UNIX** esistono diverse varianti di shell. Le più importanti sono la **shell Bourne (sh)**, la **shell C (csh)** e la **shell Korn (ksh)**. Tutte si comportano in modo simile in uso interattivo ma differiscono nella sintassi e nell'efficienza come linguaggi di **scripting**.

La **shell Bourne**, sviluppata da AT&T, è la shell standard per la gestione del sistema. È nota per essere **concisa** e **veloce**, e il suo **prompt** predefinito è \$.

La **shell C (csh)**, sviluppata a **Berkeley**, aggiunge funzionalità avanzate come **storia dei comandi**, **alias**, **completamento automatico**, **aritmetica interna** e **controllo avanzato dei job**. È stata apprezzata per l'interazione utente, ma per **scripting** è preferita la Bourne per la sua semplicità. Il prompt della C shell è %.

La **shell Korn (ksh)**, sviluppata da **David Korn** nei laboratori AT&T, è una **estensione compatibile** della Bourne, con aggiunte come **funzioni**, **alias**, **wildcard**, **controllo dei job** e **aritmetica interna**. È quasi totalmente compatibile con sh, e anche per lei il prompt è \$. Una versione open source chiamata **pdksh** è disponibile su più piattaforme, inclusa Linux.

La Shell di Linux

In **Linux**, si ha accesso a shell e strumenti **GNU**. La shell predefinita è la **GNU Bourne Again Shell (bash)**, un'estensione della Bourne. Bash è molto diffusa sia per uso interattivo che per scripting, ed è **conforme allo standard POSIX**. Il suo prompt predefinito è **\$**.

Altre shell usate in Linux includono la **tcsh** (una variante della C shell, con caratteristiche aggiuntive, il cui prompt è **>**) e la **zsh**, che unisce le funzionalità di Bourne, Korn e TC in una shell moderna.

Infine, per sapere quali shell sono installate in un sistema Linux, si può esaminare il file:

```
cat /etc/shells
```

Usi della shell: interprete dei comandi

Quando la shell viene usata in modalità **interattiva**, una delle sue principali funzioni è quella di **interpretare i comandi** immessi al **prompt**. La shell analizza la riga di comando, la suddivide in parole (i cosiddetti **token**) separate da spazi, valuta la presenza di **metacaratteri speciali**, ed esegue il comando una volta completata l'analisi. In questo contesto, la shell non si limita all'elaborazione dei comandi, ma gestisce anche le operazioni di **I/O** e l'**esecuzione in background** dei processi.

Oltre al suo utilizzo interattivo, la shell può funzionare anche come un **linguaggio di programmazione interpretato**. I cosiddetti **script di shell** sono file che contengono una sequenza di comandi e vengono scritti tramite un **editor di testo**. Questi script possono includere **variabili**, **test condizionali**, **cicli** e altri costrutti fondamentali della programmazione. La loro esecuzione non richiede compilazione: la shell interpreta ogni riga dello script come se fosse digitata direttamente dall'utente.

Startup del sistema e shell di Login

All'avvio del sistema, il primo processo eseguito è **init**, con **PID 1**, che ha il compito di inizializzare il sistema e lanciare gli altri processi necessari a configurare il terminale. Vengono definiti i file standard di input, output ed error associati al terminale. Terminata questa fase, viene visualizzato il **prompt dei comandi**, che richiede l'inserimento di **nome utente e password**. Il programma **/bin/login** si occupa di verificare l'identità dell'utente consultando il file **/etc/passwd**. Una volta autenticato, vengono impostate le **variabili d'ambiente** come **HOME**, **SHELL**, **USER**, **LOGNAME** e **PATH**, quest'ultima utile per localizzare i comandi più usati.

Conclusa la procedura di login, il sistema lancia la **shell di login**, ovvero il programma specificato nel file **passwd**. La shell controlla se nella home dell'utente sono presenti file di **inizializzazione** (come **.bashrc**, **.profile** ecc.) e li esegue, permettendo così la **personalizzazione dell'ambiente**.

Parsing della riga di comando

Ogni volta che un utente inserisce un comando, la shell esegue il **parsing della riga di comando**, suddividendola in **token** separati da spazi. Quindi, controlla se il primo token corrisponde a un **comando built-in** oppure a un file eseguibile situato su disco. In caso di comando interno, viene eseguito direttamente. Altrimenti, la shell cerca nelle directory specificate dalla variabile **PATH** per determinare la posizione del programma. Se il file viene trovato, viene creato un **nuovo processo** tramite **fork** e viene eseguito. Dopo l'esecuzione, la shell attende la **terminazione del processo** e riporta eventualmente lo **stato di uscita**.

Tipi di comandi

Quando in **UNIX** viene eseguito un comando, esso può corrispondere a un **alias**, una **funzione**, un **comando built-in** oppure a un **programma eseguibile** memorizzato su disco. Gli **alias** sono semplici abbreviazioni di comandi, mentre le **funzioni** sono gruppi di comandi organizzati in routine che vengono definite nella memoria della shell. I comandi **built-in** sono invece funzioni predefinite all'interno della shell stessa, e i **programmi eseguibili** sono file reali sul disco.

La **shell** determina in quale forma deve interpretare il comando, e per farlo esegue un controllo in ordine: prima verifica se è un **alias**, poi se è una **parola chiave**, quindi una **funzione**, poi un **comando built-in** e infine se è un **programma eseguibile**. Per individuare questi ultimi, la shell utilizza la variabile di ambiente **PATH**, cercando nelle directory elencate in essa.

I Processi e la shell

Un **processo** è un programma in esecuzione, identificato da un **PID** (Process ID), che il **kernel** del sistema gestisce e controlla. Ogni processo include dati, **stack**, **registri** e tutte le risorse necessarie per l'esecuzione. La **shell** stessa, una volta avviata, diventa un processo, appartenente a un gruppo identificato da un **pid del gruppo**. Solitamente solo un gruppo alla volta ha il controllo del terminale (è in **foreground**).

Per monitorare i processi in esecuzione si può utilizzare il comando **ps**, che, con diverse opzioni (come `ps aux`), mostra una lista dei processi attivi. In alternativa si può usare **pstree**, che visualizza l'albero dei processi partendo dalla radice (di solito il processo **init**). Se si specifica un utente, **pstree** mostra solo i processi di quell'utente.

La **sintassi** di base del comando `ps` è:

```
ps [options]
```

Le opzioni più comuni includono:

- **a**: mostra anche i processi degli altri utenti
- **x**: mostra processi non associati al terminale
- **e**: mostra tutti i processi

Tra le informazioni che `ps` può restituire ci sono:

- **PID** (ID del processo)
- **TTY** (terminale associato)
- **STAT** (stato del processo)
- **TIME** (tempo CPU utilizzato)
- **CMD** (comando eseguito)

Una volta effettuato il **login**, la shell eredita una **identità**, che include: uno **UID** (user ID), uno o più **GID** (group ID), e degli **EUID/EGID** (effective user/group ID). Con il comando:

```
id
```

è possibile visualizzare tutte queste informazioni.

Proprietà e Permessi

Ogni file in UNIX/Linux ha associati dei **permessi** che ne regolano l'accesso in **lettura (r)**, **scrittura (w)** e **esecuzione (x)**. I permessi sono rappresentati da **nove bit**, suddivisi in tre gruppi: per il **proprietario del file**, per il **gruppo** e per **tutti gli altri**.

Ad esempio, con:

```
ls -l prova.c
```

si ottiene una stringa come `-rw-r--r--`, che indica i permessi del file per ogni gruppo di utenti.

Per modificare questi permessi, l'utente deve essere il **proprietario** del file. I permessi possono anche essere rappresentati in forma **ottale**, dove ogni cifra è una combinazione binaria di `r`, `w`, `x`. Ecco la tabella corrispondente:

Ottale	Binario	Permessi
0	000	Nessuno
1	001	--x
2	010	-w-
3	011	-wx
4	100	r--
5	101	r-x
6	110	rw-
7	111	rwX

Maschera di creazione dei file

Quando un file viene creato in un sistema UNIX o Linux, i **permessi di default** che gli vengono assegnati dipendono dalla **maschera** (o **umask**) attiva nel processo che lo genera.

Questa **maschera di bit** viene applicata ai permessi completi di default per sottrarne determinati diritti.

I **permessi** sono quindi determinati dal processo di creazione del file, e i processi **figli** ereditano la **umask** dal processo **genitore**. L'utente può modificare il valore della **umask** manualmente con il comando `umask`, oppure impostandolo all'interno dei file di inizializzazione della shell, come `/etc/profile`.

All'inizio, il valore di **umask** è tipicamente `000`, che equivale a concedere permessi `777` (ovvero **rwXrwXrwX**) per le directory e `666` (**rw-rw-rw-**) per i file. Tuttavia, nella maggior parte dei sistemi operativi questo valore viene modificato in `022` tramite il programma `/bin/login` o file di inizializzazione. Questo valore `022`, quando sottratto dai permessi di default, porta a `755` (**rwXr-xr-x**) per le directory e `644` (**rw-r--r--**) per i file. In pratica, la **umask** stabilisce quali permessi **non** saranno concessi a file e directory appena creati.

Cambiare permessi e proprietà

Per modificare manualmente i **permessi** dei file e delle directory si utilizza il comando `chmod`. Questo comando consente di assegnare o rimuovere i permessi di **lettura (r)**, **scrittura (w)** ed **esecuzione (x)** a diversi gruppi di utenti. Soltanto il **proprietario** del file, o il **superutente**, ha la possibilità di modificarli. Ogni file può avere un solo proprietario e può appartenere a un solo gruppo, ma il proprietario può regolare i permessi del proprio gruppo per abilitare o meno l'accesso. È possibile consultare i permessi di un file digitando `ls -l nomefile`.

Alcuni esempi pratici chiariscono l'uso del comando `chmod`:

```
$ chmod 755 file
$ ls -l file
-rwxr-xr-x 1 staiano staiano ...
```

Successivamente, se si volesse aggiungere il permesso di scrittura al gruppo:

```
$ chmod g+w file
$ ls -l file
-rwxrwxr-x 1 staiano staiano ...
```

Per togliere i permessi di lettura ed esecuzione a gruppo e altri:

```
$ chmod go-rx file
$ ls -l file
-rwx-w---- 1 staiano staiano ...
```

Infine, per impostare la lettura per tutti:

```
$ chmod a=r file
$ ls -l file
-r--r--r-- 1 staiano staiano ...
```

Sintassi di *chmod*

La **sintassi generale** del comando `chmod` è:

```
chmod [opzioni] XYZ nome_file
```

dove X indica a quali utenti si vogliono applicare le modifiche (a per tutti, g per il gruppo, u per l'utente, o per gli altri), Y rappresenta l'azione (+ per aggiungere, - per togliere), e Z specifica il tipo di permesso (r per lettura, w per scrittura, x per esecuzione).

Oltre a `chmod`, esiste anche il comando `chown`, che serve per **modificare il proprietario o il gruppo** a cui appartiene un file. Su **sistemi Linux**, solo il **superutente** può eseguire questa operazione. Nei **sistemi UNIX**, sia il **proprietario** del file che il **superutente** possono modificare le proprietà.

Un esempio tipico con `chown` è il seguente:

```
$ ls -l filetest
-rw-rw-r-- 1 staiano staiano ...
```

```
$ chown root filetest
chown: filetest: Operation not permitted
```

```
$ su root
Password:
```

```
# chown root filetest
# ls -l filetest
-rw-rw-r-- 1 root staiano ...
```

```
# chown root:root filetest
# ls -l filetest
-rw-rw-r-- 1 root root ...
```

Come mostrato, solo cambiando utente a **root** (`su root`) è possibile modificare la proprietà del file. Questi comandi permettono di **gestire con precisione** la sicurezza e l'accesso ai file nel sistema.

Comando *su*

Il comando **su** (che significa "substitute user" o "set user") viene utilizzato per cambiare utente nella shell. Se invocato senza parametri, consente di diventare l'**amministratore di sistema** (root). Ad esempio, scrivendo semplicemente `su`, il sistema chiederà la password dell'utente root e, se corretta, garantirà l'accesso con privilegi amministrativi. Allo stesso modo, si può accedere come altro utente specificando il nome, ad esempio `su mario` per diventare l'utente "mario", oppure `su root` per diventare root in modo esplicito. In tutti i casi, viene richiesta la password dell'utente specificato.

Directory di lavoro

A ogni utente che effettua il login è assegnata una **directory di lavoro**, detta anche **home directory**, nel filesystem. Questa directory è ereditata da tutti i processi generati dalla shell. Se un processo figlio cambia directory, il processo padre (la shell) non ne risente. La directory può essere modificata con il comando **cd**, che è un **comando built-in** della shell. Poiché è integrato nella shell stessa, non viene eseguito come processo separato ma direttamente nel contesto del processo corrente. Se viene avviata una nuova shell (ad esempio con `bash`), essa può cambiare directory senza influenzare la directory della shell originale.

Variabili

La shell consente anche di definire **variabili**, che possono essere **locali** o di **ambiente**. Le variabili locali sono visibili solo nella shell in cui sono definite e non vengono trasmesse ai processi figli. Le variabili di ambiente, invece, vengono ereditate da tutti i processi generati dalla shell, fino anche ai "nipoti". Alcune variabili di ambiente sono automaticamente impostate dal sistema (es. tramite `/bin/login`), altre possono essere create nei file di inizializzazione della shell o direttamente a riga di comando. È importante sapere che una variabile definita in una shell figlia non si propaga a ritroso verso la shell padre.

Per visualizzare le variabili locali e di ambiente, si può usare il comando **set**, mentre per visualizzare solo le variabili di ambiente si può usare **env**.

Redirezione

La redirezione in UNIX si basa sull'uso dei **file descriptor**, un meccanismo attraverso il quale il **kernel** gestisce tutto l'I/O, inclusi file, pipe e socket. Ogni **file descriptor** è rappresentato da un intero non firmato, che agisce come indice in una tabella interna mantenuta dal kernel. Questa tabella serve per tenere traccia dei file aperti da ciascun processo.

Quando un processo figlio viene generato, eredita la **tabella dei file descriptor** dal processo padre. I primi tre file descriptor sono riservati al terminale: il **0** rappresenta lo **standard input**, l'**1** lo **standard output** e il **2** lo **standard error**. Quando si apre un nuovo file, il primo descriptor disponibile è in genere il **3**, che verrà assegnato al file appena aperto.

La redirezione è il processo con cui si associa uno di questi file descriptor a un file, invece che al terminale.

La **shell** gestisce questa operazione chiudendo il file descriptor associato, ad esempio, all'output e assegnandolo a un nuovo file. In modo simile, se si ridirige lo standard input, la shell chiude il file descriptor **0** e lo ricollega a un altro file.

Vediamo alcuni esempi pratici di redirezione.

Il comando:

```
$ who > file
```

esegue una redirezione dell'output del comando **who**, che normalmente sarebbe visualizzato nel terminale, verso un file chiamato **file**. In questo modo, il risultato del comando viene salvato in quel file.

Un altro esempio è:

```
$ cat file1 file2 >> file3
```

che concatena **file1** e **file2** e aggiunge il risultato in coda al file **file3**, senza sovrascrivere il contenuto preesistente.

Infine:

```
$ find / -name file -print 2> errors
```

esegue il comando **find**, cercando un file nella directory radice. Se il comando genera un errore, questo non verrà mostrato nel terminale ma sarà salvato nel file **errors**, grazie alla redirezione dello **standard error**.

A livello visivo, quando viene eseguito un comando con redirezione, ad esempio:

```
$ grep Tom datafile > temp
```

la **shell padre** (PID 124) con lo **stdout** associato allo **schermo** avvia un processo figlio (PID 125). Nel figlio, lo **stdout** viene associato al file **temp**, mentre lo **stdin** resta legato alla tastiera e lo **stderr** continua ad andare allo schermo.

Pipe

Le **pipe** rappresentano una delle prime forme di **comunicazione tra processi (IPC)** nei sistemi **UNIX**. Servono a permettere a due o più processi di **comunicare tra loro**, trasferendo l'**output di un comando** come **input per un altro comando**. Questo flusso di dati è **monodirezionale**, cioè i dati scorrono in una sola direzione: dal comando a sinistra della pipe a quello a destra.

Quando un utente utilizza la **pipe** (indicata con il simbolo `|`), la **shell** si occupa di creare un meccanismo interno che collega i processi coinvolti. Per farlo, la shell **chiude e apre i descrittori di file** dei processi, ma invece di associarli a normali file su disco, li **collega tra loro tramite un buffer del kernel**, creando una vera e propria **pipe** grazie a una chiamata di sistema.

Una **pipe**, in termini pratici, è un **buffer gestito dal kernel**. Questo buffer può essere letto e scritto contemporaneamente da due processi. Quando un comando produce dati, essi vengono **inviati al buffer**, e appena il buffer è pieno o il comando ha terminato la sua esecuzione, il secondo comando può iniziare a **leggere i dati dal buffer**.

Il **kernel** si occupa di **sincronizzare i processi**, garantendo che un processo scriva solo quando l'altro è pronto a leggere, e viceversa. In questo modo, si evita l'utilizzo di file temporanei sul disco per lo scambio di dati tra comandi.

Un esempio pratico di utilizzo di una pipe è il comando:

```
$ who | wc
```

In questo caso, il comando `who` produce una lista degli utenti connessi al sistema, e questa lista viene **passata direttamente al comando `wc` (word count)** tramite la pipe. Il comando `wc` poi conta il numero di righe, parole e caratteri ricevuti da `who`.

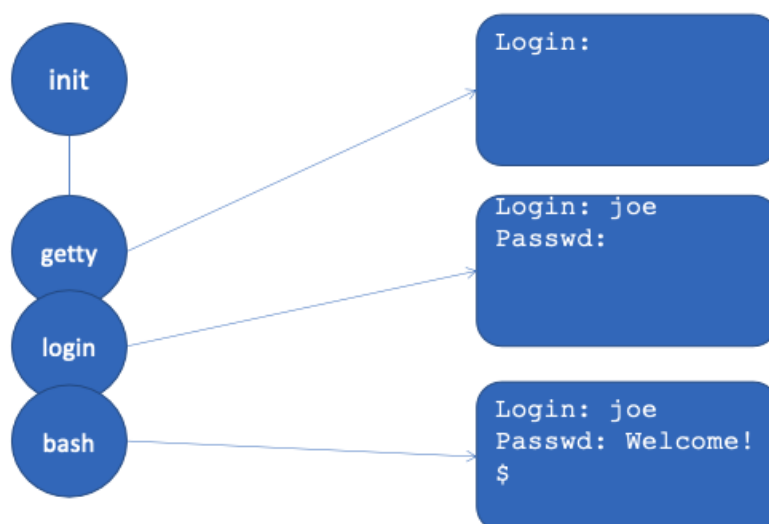
Senza l'uso della pipe, per ottenere lo stesso risultato sarebbe stato necessario scrivere:

```
$ who > tempfile  
$ wc tempfile  
$ rm tempfile
```

Questa sequenza usa un **file temporaneo** per memorizzare l'output di `who`, che viene poi letto da `wc` e infine eliminato. La pipe rende tutto questo più semplice ed efficiente, eliminando la necessità del file intermedio.

Bourne Again Shell

La **Bourne Again Shell (Bash)** rappresenta l'evoluzione della shell Bourne ed è la shell predefinita in molte distribuzioni Linux. Al momento dell'avvio del sistema, il primo processo che viene eseguito dal kernel è chiamato **init**, che riceve il PID 1. Questo processo ha il compito di avviare l'ambiente di sistema, ed è proprio da `init` che viene generato un processo secondario chiamato **getty**.



Il processo **getty** apre l'accesso al terminale, specificando quali sono le sorgenti da cui provengono l'input e verso cui vanno l'output e gli errori standard. Questo processo è quello che mostra il prompt per effettuare il login. Successivamente viene eseguito il programma **/bin/login**, il quale si occupa di richiedere una password all'utente, verificarla e, se corretta, configurare un ambiente di lavoro iniziale. Dopo questa fase, il programma di login avvia la shell di login, tipicamente **/bin/bash**.

Una volta avviata la shell, essa esegue i comandi presenti nel file di configurazione globale **/etc/profile**. Poi, all'interno della directory home dell'utente, la bash esegue altri file di inizializzazione, come ad esempio **.bash_profile**. Al termine di questa sequenza, viene lanciato il file **.bashrc**, che contiene ulteriori comandi e configurazioni dell'ambiente utente. Dopo tutte queste operazioni, sulla schermata del terminale compare il prompt predefinito, rappresentato dal simbolo del dollaro \$, che indica che la shell è pronta a ricevere comandi.

Bash mette anche a disposizione il comando **set**, un comando interno (o built-in) che permette di modificare il comportamento della shell attraverso delle opzioni. Il comando `set -o` viene usato per attivare una specifica opzione, mentre `set +o` la disattiva. Esistono anche forme abbreviate, dove si utilizza `set -x` per attivare e `set +x` per disattivare un'opzione associata alla lettera x.

Un esempio pratico può essere `set -o allexport`, oppure la sua forma abbreviata `set -a`, che serve per fare in modo che tutte le variabili di ambiente definite nella sessione vengano automaticamente esportate anche alle sottoshell. Questo tipo di configurazione è molto utile quando si vogliono propagare variabili definite in uno script o in una sessione interattiva a processi figli generati dalla stessa shell.

I comandi della shell

Comando history

Quando si parla dei **comandi della shell**, è importante sapere che tutti i comandi digitati dall'utente vengono memorizzati in un file nascosto, all'interno della propria **directory home**. Questo file è chiamato **.bash_history**. Il punto all'inizio del nome indica che si tratta di un file nascosto. Ad esempio, un utente chiamato "mau" avrà lo storico dei suoi comandi nel file `/home/mau/.bash_history`.

Per visualizzare questo file nascosto si può utilizzare il comando `ls -a`. Per leggerne il contenuto, è sufficiente usare il comando `cat .bash_history`, oppure, in modo più diretto, il comando `history`, che ha lo stesso effetto.

Se si digita `history`, ad esempio si otterrà un elenco numerato di comandi eseguiti, come:

```
1 ls
2 cd miadir
3 cat pippo
4 ls -lai
```

Digitando `!3` si eseguirà il terzo comando dell'elenco, quindi `cat pippo`. È anche molto utile il tasto **TAB**, che permette di espandere nomi di file o comandi incompleti. Se, ad esempio, si desidera eseguire il comando `touch`, ma non ci si ricorda come si scrive completamente, basta digitare `to` e premere **TAB** per vedere tutti i comandi che iniziano con quella sequenza.

Altri comandi

Alcuni comandi molto utili per ottenere informazioni sul sistema sono `date` per sapere la data e l'ora, `who` per sapere chi è connesso, `cal` per visualizzare il calendario e soprattutto `man` per consultare la documentazione dettagliata dei comandi disponibili. Questo comando restituisce pagine organizzate in sezioni che descrivono nel dettaglio sintassi, parametri, file correlati, messaggi di errore noti, bug e persino esempi pratici.

Ogni pagina del manuale è organizzata in **sezioni numerate**. Ad esempio, la sezione 1 raccoglie i comandi da terminale, la 2 le chiamate di sistema, la 3 le funzioni di libreria, e così via. Se non si specifica la sezione, `man` mostrerà la prima che trova. È comunque possibile indicare la sezione desiderata con l'opzione `-s`. Ad esempio, `man -s 2 kill` consente di accedere alla pagina della sezione 2 relativa al comando `kill`, che riguarda la chiamata di sistema.

Un altro esempio: `man who` mostra la documentazione relativa al comando `who`, mentre `man -s 2 intro` restituisce l'introduzione alla sezione 2 del manuale. Ogni sezione o sottosezione inizia infatti con una pagina introduttiva chiamata proprio `intro`.

Visualizzare il contenuto di un file

Per leggere o concatenare file tramite la shell, è possibile utilizzare il comando **cat**. Quando si scrive `cat file1 file2`, il contenuto dei due file viene unito e visualizzato nello standard output, tipicamente il terminale. Scrivendo solo `cat file`, si ottiene semplicemente la visualizzazione del contenuto di un singolo file. Inoltre, digitando semplicemente `cat` senza argomenti, la shell si mette in attesa di input da tastiera, che verrà copiato direttamente sull'output standard fino alla ricezione di un'interruzione.

Alias

Un **alias** è un'abbreviazione o una scorciatoia per un comando, creata dall'utente all'interno della shell Bash. Risulta particolarmente utile nei casi in cui un comando sia lungo, complesso o con una sintassi difficile da ricordare. Gli alias definiti nella riga di comando non vengono trasmessi alle subshell, per questo motivo è buona pratica definirli nel file `.bashrc`, che viene eseguito automaticamente ogni volta che si apre una nuova shell.

Elencare gli alias

Per visualizzare tutti gli alias definiti all'interno dell'ambiente corrente della shell, si utilizza semplicemente il comando `alias`. Questo restituisce un elenco di abbreviazioni con la loro corrispondenza ai comandi completi. Ad esempio, può essere visualizzato un alias come `ll='ls -aF'`, il che significa che ogni volta che si digita `ll`, verrà eseguito `ls -aF`.

Creare gli alias

La creazione di un alias avviene specificando innanzitutto il nome dell'abbreviazione e successivamente il comando associato. Per esempio, `alias m=more` definisce `m` come scorciatoia per il comando `more`. È possibile anche creare alias con comandi più complessi come `alias lF='ls -aF'` oppure `alias r='fc -s'`, utile per ripetere l'ultimo comando modificandolo.

Cancellare gli alias

Se un alias non è più necessario, lo si può eliminare con il comando **unalias**, seguito dal nome dell'alias. Ad esempio, `unalias mroe` rimuove l'abbreviazione `mroe`. Se invece si desidera temporaneamente ignorare un alias, è sufficiente precedere il nome del comando con una barra rovesciata, ad esempio `\ls`, per eseguire il comando originale `ls` senza che l'alias venga interpretato.

Comandi frequenti per la gestione delle directory

Iniziamo con le directory principali. La directory `/dev` contiene i driver dei dispositivi hardware. Le directory `/bin` e `/usr/bin` ospitano i comandi standard di Linux, mentre le librerie essenziali si trovano in `/lib` e `/usr/lib`. I file di log e di configurazione si trovano in `/var` e `/etc`, rispettivamente. I comandi personalizzati o installati successivamente sono memorizzati in `/usr/local/bin`. Il software commerciale viene solitamente collocato in `/opt`, mentre i file temporanei risiedono in `/tmp`. Infine, i comandi di sistema riservati all'amministratore sono contenuti in `/sbin` e `/usr/sbin`.

Primi comandi

La directory root, indicata con il simbolo `/`, è il punto di partenza dell'intero file system. Ogni utente ha una propria directory personale, situata sotto `/home`, con il proprio nome utente. Per conoscere quale utente è attualmente connesso, si può utilizzare il comando `whoami`. Per ottenere l'elenco dei file presenti nella directory corrente si usa `ls`. Se si desidera cambiare directory e

accedere, ad esempio, a quella denominata Desktop, si può digitare `cd Desktop`. Per tornare indietro alla directory precedente si utilizza `cd ...`

touch

Il comando `touch` viene utilizzato per modificare l'istante di accesso o di modifica di un file. Se il file indicato non esiste, viene creato automaticamente. Ad esempio, posizionandosi nella directory Desktop e digitando `touch pluto`, si creerà un nuovo file chiamato "pluto". Questo file sarà poi visibile con il comando `ls`. Il comando `touch` accetta anche delle opzioni, come `-a`, per aggiornare solo l'ora di accesso, oppure `-c`, per modificare l'ora di modifica del file.

Copia di un file: cp

Il comando `cp` serve per copiare un file da una directory a un'altra. Ad esempio, scrivendo `cp pluto prova`, se "prova" è una directory, il file "pluto" verrà copiato al suo interno. Se invece si scrive `cp pluto pluto1`, verrà creato un nuovo file "pluto1" con lo stesso contenuto di "pluto".

Spostare un file: mv

Per spostare o rinominare un file si utilizza il comando `mv`. Scrivendo `mv pluto pippo`, il file "pluto" viene rinominato in "pippo". Se si desidera spostare il file in un'altra directory, come "agenda", il comando sarà `mv pluto agenda`. Questo comando è valido anche per lo spostamento di directory intere.

Cancellare un file: rm

La rimozione di file o directory si effettua con il comando `rm`. Per cancellare un singolo file, come "pluto", si scrive `rm pluto`. Per eliminare una directory e tutto il suo contenuto, si usa `rm -r agenda`.

Cercare file: find

Per cercare file o directory in base al nome, alla data di creazione o alla dimensione, si utilizza il comando `find`. Ad esempio, `find . -name pluto` cerca un file chiamato "pluto" nella directory corrente. Se si vuole trovare file creati negli ultimi sette giorni si può usare `find . -ctime 7`, mentre `find . -size nKB` trova tutti i file più grandi di una certa dimensione.

sort

Infine, per ordinare alfabeticamente i contenuti di un file, si usa il comando `sort`. Ad esempio, `sort elenco.txt` stampa le righe del file in ordine alfabetico. Per ottenere un ordine inverso, si può scrivere `sort -r elenco.txt`.

Inviare segnali ad un processo

kill

Il comando `kill` è usato per mandare un segnale a un processo identificato con il suo PID (process id). La sintassi è: `kill -s tipo_segno PID`. Il segnale viene definito attraverso un nome, come ad esempio `SIGKILL`, oppure un numero. Il nome

del segnale non è sensibile alla differenza tra maiuscole e minuscole e può essere indicato anche senza il prefisso `SIG`. Se non viene indicato il tipo di segnale da inviare, si intende `SIGTERM` (terminazione standard).

Alcuni esempi mostrano l'utilizzo del comando. Digitando `kill -l`, viene elencata tutta la scelta dei segnali disponibili in una tabella. Per fermare un processo si può usare: `kill -s stop PID_number`. Questo invia un segnale di stop al processo identificato con quel numero. Per riprendere un processo fermato si può digitare: `kill -s cont PID_number`. Il comando può essere digitato anche da un'altra shell.

Esempio: kill

Si supponga di avere i seguenti processi in esecuzione:

```
31210 pts/2 00:00:00 gedit
31218 pts/2 00:00:00 vim
31226 pts/2 00:00:00 man
31229 pts/2 00:00:00 sh
31230 pts/2 00:00:00 sh
31235 pts/2 00:00:00 less
```

Se si vuole fermare il processo `gedit` per poi riavviarlo in un secondo momento, occorre digitare: `kill -s stop 31210`. Successivamente, usando il comando `jobs`, sarà possibile visualizzare che il processo `gedit` è stato fermato.

Per riavviare `gedit`, si può usare: `kill -s cont 31210`. Anche in questo caso il comando `jobs` mostra che `gedit` è ritornato in esecuzione.

Esempio di output del comando `jobs`:

```
[12] Stopped  vim
[13] Stopped  gedit &
[14] Stopped  more
```

Dopo il `kill -s cont 31210`, lo stato può diventare:

```
[13] Running  gedit &
```

Processi foreground

Tutti i processi avviati da shell sono eseguiti in foreground, ovvero assumono il controllo del terminale. Tuttavia, è possibile intervenire sulla modalità di esecuzione in diversi modi. Digitando `Ctrl+Z` si sospende un processo foreground in esecuzione. Il comando `fg` ripristina il processo in foreground. Il comando `bg` ripristina il processo in background. Il comando `jobs` mostra tutti i processi lanciati dalla shell che sono in background.

Ad esempio, se si esegue il comando `vi` (editor di testo) e si digita `Ctrl+Z` per sospenderlo, si può poi digitare `fg` per riportarlo in esecuzione foreground.

Processi background

Se si digita un comando seguito dal simbolo `&`, il comando sarà eseguito in background. Il controllo sarà subito restituito alla shell senza attendere che il comando termini. La sintassi è del tipo: `comando argomenti &`. Ad esempio: `gedit &`.

Esempio background

Se si digita `gedit &`, viene avviato GEDIT in background e il terminale visualizza qualcosa come `(gedit:31210)` indicando il PID del processo.

Digitando il comando `jobs`, si può vedere che `gedit` è in esecuzione in background. L'output sarà simile al seguente:

```
[1] Stopped  man createuser (wd: /home/mario)
[2] Stopped  more = pluto   (wd: /home/mario)
[3] Stopped  man more      (wd: /home/mario)
[4] Stopped  more = pluto   (wd: /home/mario)
[5] Stopped  more = pluto   (wd: /home/mario)
[6] Stopped  more '=' pluto (wd: /home/mario)
[7] Stopped  man more      (wd: /home/mario)
[8] Stopped  more '=' pluto (wd: /home/mario)
[9] Stopped  more elenco.txt
[10] Stopped more -2 elenco.txt
[11] Stopped  man sort
[12]+ Stopped vim
[13] Running  gedit &
```

Link

I **link** sono strumenti che permettono di accedere a file o directory attraverso delle scorciatoie, utili in particolare quando i file vengono **usati frequentemente**, **condivisi** o **acceduti con nomi diversi**. Grazie ai link, è possibile creare **più punti di accesso** a uno stesso contenuto, mantenendo comunque **un'unica copia delle informazioni**.

Tipologie di link

Esistono due **tipi principali di link**: i **soft link** (o collegamenti simbolici) e gli **hard link**. I **soft link** funzionano in modo simile ai collegamenti di **Windows**: si tratta di file speciali che contengono il **percorso (path)** del file collegato. Quando accediamo al file tramite un soft link, il sistema segue quel percorso per recuperare i dati. Tuttavia, se il file originario viene cancellato, il soft link resta, ma non punta più a nulla: si parla allora di **dangling pointer**, e qualsiasi tentativo di accesso genera un errore. Questo tipo di link comporta un **maggior uso di memoria** e può essere **più lento** nell'accesso.

Gli **hard link**, invece, non creano un nuovo file ma un nuovo **nome** che punta **allo stesso i-node** del file originale, condividendone direttamente i dati fisici. I due nomi risultano **equivalenti**: modificando uno, si modifica anche l'altro, e la cancellazione di uno dei due non elimina i dati fino a quando **tutti gli hard link** associati a quell'i-node non vengono rimossi. Questo approccio è più **efficiente in termini di memoria** e garantisce una maggiore **robustezza nell'accesso**.

Nel confronto tra i due, possiamo dire che se **"nuovo" è un soft link a "vecchio"**, posso modificare i dati da entrambi, ma la cancellazione di "vecchio" rende il soft link inutilizzabile. Se invece **"nuovo" è un hard link a "vecchio"**, i dati restano accessibili da entrambi finché almeno uno dei link esiste. Inoltre, se si ricrea un file con lo stesso nome dopo averlo eliminato, questo **non condivide più dati** con il vecchio.

Esempio di creazione link

Creando un file con `touch`, è possibile creare un hard link con `ln` e un soft link con `ln -s`. Il comando `ls -li` evidenzia che **gli hard link condividono lo stesso i-node**, mentre i soft link ne hanno uno differente e visualizzano il percorso del file collegato. Nei soft link, i **permessi** indicati riguardano il link stesso, non il file di destinazione.

